

Class 6: R Functions

Madina Khorami (PID: 5185)

Table of contents

Background	1
1. A First Function	1
2. A <code>generate_dna()</code> Function	3
3. Write a <code>generate_protein()</code> function	5
4. Generate random protein sequences of length 6 to 13	6
5. Are our peptides “unique in nature”?	7
6. Connecting your findings to immunology	8

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- Input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

1. A First Function

Here we create a function to add some number. Let's call it `add()`.

Input arguments can be either “**required**” or “**optional**”. The latter have fall-back default values that will be used if the user does not specify them.

Q1a. Your first version of the function should add two input numbers together.
For example, `add(4,7)` should return 11.

```
add <- function(x, y=0) {  
  x + y  
}
```

Can we use our new function.

```
add(4,7)
```

```
[1] 11
```

Q1b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`.

```
add <- function(x, y=0) {  
  sum(x) + y  
}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2.

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

```
add <- function(...) {  
  return(sum(...))  
}
```

```
add(1,2,3, -4)
```

```
[1] 2
```

2. A generate_dna() Function

Write a function generate_dna() that returns a random DNA sequence of a length specified by the user.

A useful function here is the base R sample() function.

```
sample(1:5, size = 55, replace=TRUE)
```

```
[1] 4 2 1 2 2 1 5 1 2 5 1 2 4 1 5 2 5 4 3 4 4 4 2 3 4 1 2 4 3 4 5 1 2 3 2 5 2 4
[39] 4 5 2 3 4 1 3 2 4 4 2 4 4 5 1 1 1
```

Q2a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return “A”, “T”, “T”, “G”, “A”, “C”.

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”....

```
## # sample() Randomly selects 10 nucleotides (A, C, G, T) allowing repeat to create a DNA s
```

```
sample(x=c("A","C","G","T"), size = 10, replace = TRUE)
```

```
[1] "A" "G" "T" "C" "T" "G" "A" "A" "G" "G"
```

Q2a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return “A”, “T”, “T”, “G”, “A”, “C”.

```
# The code returns a random DNA sequence of a length specified by the user with a default va
```

```
generate_dna <- function(len=10) {
  sample(x=c("A","C","G","T"), size = len, replace = TRUE)
}
```

```
generate_dna()
```

```
[1] "G" "A" "A" "T" "C" "T" "A" "T" "C" "A"
```

```
generate_dna(len=20)
```

```
[1] "G" "T" "A" "G" "G" "A" "A" "C" "A" "C" "G" "G" "A" "C" "A" "T" "T" "T" "C"  
[20] "G"
```

Q2b. Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”.

Functions that could be useful are `paste()`, `if()`, `cat()`, and `return()`.

```
# Step 1: randomly generate a DNA sequence of length 'len' sample() picks from A, C, G, T and  
generate_dna <- function(len, single.element=TRUE) {  
  ans <- sample(x=c("A","C","G","T"), size = len, replace = TRUE)  
  
# Step 2: check if user wants a single combined string and paste() joins all letters into one  
  if(single.element) {  
    return(paste(ans, collapse = ""))  
  }  
  
# Step 3: if not, return the vector of individual letters no single element  
  else {  
    return(ans)  
  }  
}
```

```
generate_dna(len = 20)
```

```
[1] "ATACTCACTCGGGCCTGTAG"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

For example:

```
>len9
```

```
CGAAGGCTG
```

```

## Step 1: generate a random DNA sequence of length 'len'
## sample() picks letters from A, C, G, T with replacement.

generate_dna <- function(len, single.element=TRUE) {
  ans <- sample(x=c("A","C","G","T"), size = len, replace = TRUE)

  ## Step 2: if user wants a single string, combine all letters.

  if(single.element) {
    ans <- paste(ans, collapse="")
  }

  ## Format as FASTA with an ID line
  ## Step 3: print FASTA header line with sequence length and ">len9" format with no space betw

  cat(paste(">len", len, "\n", sep = ""))

  ##Step 4: print the DNA sequence
  cat(ans)

  ## Step 5: move to next line for clean output
  cat("\n")

  ## Step 6: return the sequence as string or vector depending on input.
  return(ans)
}

x <- generate_dna(20)

```

```

>len20
AGATACAATTAAATCTCTCG

```

3. Write a generate_protein() function

Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG". Your function should:

- Use the single-letter code for all **20** standard amino acids and no other letters (see earlier list at the beginning of this handout) and include clear comments that explain each step of your function.

```
# Step 1: make a vector of the 20 amino acid one-letter codes
generate_protein <- function(length) {
  aa <- c("G","A","V","L","I", "P", "M","F","Y","W", "S", "T", "C","N","Q","K","R","H", "D",

# Step 2: randomly choose amino acids from the vector size = length means how long the prote
  ans <- sample(x=aa, size = length, replace = TRUE)

# Step 3: join all amino acids into one single string with no space defined by collapse.
  paste(ans, collapse = "")
}
```

```
generate_protein(6)
```

```
[1] "DCMPLR"
```

4. Generate random protein sequences of length 6 to 13

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
## Loop through lengths from 6 to 13
for (l in 6:13) {

# Print a header line (like FASTA style) with the current length
# ">" is used to indicate the sequence ID

  cat(">", l, "\n", sep = "")

# Generate a protein sequence of length l and print it
  cat(generate_protein(l), "\n")
}
```

```
>6
SIGKNP
>7
WSHCWCL
>8
```

```

IIIKHVQL
>9
HDMVDPQWR
>10
DMYHSPQSDW
>11
EYLMGAGVFKV
>12
DKGCNIKCPQNI
>13
HCGNSTNMACNFA

```

5. Are our peptides “unique in nature”?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Table 1: Compared Data with NCBI

Length (aa)	% Identity	% Coverage	Unique
6	100	100	N
7	100	100	N
8	100	88	Y
9	100	78	Y
10	100	80	Y
11	100	73	Y
12	100	67	Y
13	100	62	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

Peptides start to look unique at length 8.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20ⁿ for a peptide of length n) and to the size of the known protein universe.

Short peptides are usually found in the database because the sequence space (20^L) is still small, and the known protein universe is huge so most of those combinations already exist. As length increases, 20^L becomes extremely large, so the sequence space is much bigger than the known proteins, making matches unlikely therefore more unique protein found as the length increase.

6. Connecting your findings to immunology

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

From my Q5 data, peptides start to appear unique at around length 8, since shorter peptides (6–7) almost always have exact matches in the database. This is because the sequence space (20^L) is still small for short peptides, while the known protein universe is very large, so many of those sequences already exist. As peptide length increases, 20^L grows rapidly, making it much less likely for longer peptides to match existing proteins. Based on this, the minimum length for MHC class II presentation is likely around 8 or longer. It would be a bad design choice to present very short peptides because they are not unique and could match many self proteins, increasing the risk of triggering an immune response against the body's own cells.